

# PROJECT ETHER

## v5.0 — Repository Bootstrap + Código-base

**Objetivo desta versão:** transformar o Blueprint v4.0 em uma base de código compilável, testável e preparada para receber as primeiras features jogáveis.

## 1. REGRA PRINCIPAL

A partir desta versão, o agente de desenvolvimento deve tratar este documento como contrato técnico.

Ele deve:

1. criar a solução;
2. criar os projetos;
3. configurar dependências;
4. criar a arquitetura;
5. implementar os primeiros contratos;
6. configurar PostgreSQL;
7. configurar Redis;
8. configurar Docker;
9. criar WebSocket;
10. criar testes;
11. criar seed;
12. executar build;
13. executar testes;
14. corrigir erros;
15. documentar o estado final.

### É proibido:

- inventar regra de negócio silenciosamente;
- substituir uma regra definida neste documento;
- criar microserviços prematuramente;
- colocar lógica de negócio dentro do Godot;
- confiar no cliente para validar ações;
- armazenar estado crítico apenas no Redis;

- criar sistema de monetização antes do core gameplay;
- criar conteúdo enorme antes do vertical slice;
- adicionar dependência sem justificativa.

Quando faltar uma regra:

SPEC\_GAP

Quando duas regras entrarem em conflito:

SPEC\_CONFLICT

Quando for necessária uma decisão de produto:

PRODUCT\_DECISION\_REQUIRED

O agente deve registrar o problema e continuar apenas se existir um comportamento seguro e reversível.

## 2. STACK OFICIAL

### Backend

.NET 10  
C# 14  
ASP.NET Core  
EF Core  
PostgreSQL  
Redis  
WebSockets  
OpenTelemetry

### Client

Godot 4.x  
GDScript inicialmente

A comunicação entre client e servidor deve utilizar um protocolo próprio sobre WebSocket.

Não utilizar SignalR como protocolo principal do jogo.

Motivo:

O servidor precisa controlar explicitamente:

- sequência;
- snapshots;
- deltas;
- tick;
- comandos;
- idempotência;
- interest management;
- replicação;
- versionamento de protocolo.

ASP.NET Core fornece suporte nativo ao WebSocket e permite configurar keep-alive e timeout.

### 3. ESTRUTURA DO REPOSITÓRIO

```
ProjectEther/  
├── ProjectEther.sln  
├── src/  
│   ├── Ether.Domain/  
│   ├── Ether.Application/  
│   ├── Ether.Infrastructure/  
│   ├── Ether.Contracts/  
│   ├── Ether.Api/  
│   ├── Ether.GameServer/  
│   └── Ether.Worker/  
├── tests/  
│   ├── Ether.Domain.Tests/  
│   ├── Ether.Application.Tests/  
│   ├── Ether.Infrastructure.Tests/  
│   └── Ether.Api.Tests/
```

```
|
|   └─ Ether.GameServer.Tests/
|   └─ Ether.Architecture.Tests/
|
| └─ client/
|   └─ Ether.Game/
|
| └─ content/
|   └─ maps/
|   └─ creatures/
|   └─ items/
|   └─ abilities/
|   └─ quests/
|   └─ npcs/
|
| └─ docs/
|   └─ architecture/
|   └─ protocol/
|   └─ gameplay/
|   └─ economy/
|   └─ decisions/
|
| └─ scripts/
|
| └─ docker-compose.yml
| └─ Directory.Build.props
| └─ Directory.Build.targets
| └─ global.json
| └─ .editorconfig
| └─ .gitignore
| └─ README.md
```

## 4. RESPONSABILIDADE DOS PROJETOS

### **Ether.Domain**

Somente domínio.

Pode conter:

- entities;
- aggregates;
- value objects;
- enums;
- domain events;
- domain exceptions;
- regras puras.

Não pode depender de:

EF Core

ASP.NET

Redis

PostgreSQL

Godot

HTTP

WebSocket

## **Ether.Application**

Casos de uso.

Exemplos:

CreateCharacter

SelectCharacter

MoveCharacter

AttackCreature

CastAbility

PickupItem

EquipItem

CompleteQuest

CreateParty

SendChatMessage

Depende apenas de:

Domain  
Contracts

## **Ether.Infrastructure**

Implementações técnicas:

EF Core  
PostgreSQL  
Redis  
repositories  
unit of work  
outbox  
authentication infrastructure

## **Ether.Contracts**

Contrato de comunicação.

Deve conter:

commands  
events  
DTOs  
protocol envelopes  
error contracts  
serialization contracts

Não deve conter regras de domínio.

## **Ether.Api**

Responsável por:

HTTP

authentication

account management

character management

health checks

admin endpoints

WebSocket handshake

## **Ether.GameServer**

Responsável pelo loop do mundo.

WorldLoop

GameSessionManager

CommandProcessor

InterestManager

ReplicationManager

AI

Combat processing

Movement processing

## **Ether.Worker**

Processos assíncronos:

Outbox

notifications

analytics

cleanup

scheduled jobs

market expiration

world maintenance

## 5. DOMAIN PRIMITIVES

Criar:

```
public readonly record struct AccountId(Guid Value);
public readonly record struct CharacterId(Guid Value);
public readonly record struct ItemInstanceId(Guid Value);
public readonly record struct CreatureInstanceId(Guid Value);
public readonly record struct QuestId(Guid Value);
public readonly record struct GuildId(Guid Value);
public readonly record struct PartyId(Guid Value);
public readonly record struct MapId(int Value);
```

Todos devem possuir:

- validação;
- equality;
- conversão explícita quando necessário;
- Empty/None quando aplicável.

## 6. GOLD

Nunca utilizar `decimal` ou `double` para dinheiro do jogo.

```
public readonly record struct Gold(long Amount)
{
    public Gold
    {
        if (Amount < 0)
            throw new DomainException("Gold cannot be negative.");
    }
}
```

Operações:

Add

Subtract

CanAfford

Toda alteração financeira deve gerar:

Transaction

## 7. WORLD POSITION

```
public readonly record struct WorldPosition(  
    MapId MapId,  
    int X,  
    int Y);
```

MVP:

2D tile grid

Não utilizar float para posição lógica do personagem.

Interpolação visual pertence ao client.

## 8. CHARACTER

Aggregate root:

Character

Campos:

CharacterId

AccountId

Name

Class

Level

Experience

## Attributes

Health

Mana

Position

State

CreatedAt

UpdatedAt

Estados:

Offline

Loading

InWorld

Combat

Dead

Respawning

DisconnectGrace

Métodos:

MoveTo()

ApplyDamage()

Heal()

GrantExperience()

LevelUp()

Die()

Respawn()

Equip()

Unequip()

Nenhum setter público para mutação crítica.

## 9. ATTRIBUTES

MVP:

Strength

Dexterity

Intelligence  
Vitality  
Willpower

Cada atributo deve permitir:

Base  
Allocated  
Equipment  
Buff  
Debuff  
Final

Fórmula:

Final =  
Base  
+ Allocated  
+ Equipment  
+ Buff  
- Debuff

## 10. CLASSES

### MVP

Somente:

Warrior

Não implementar Ranger e Arcanist ainda.

Mas o domínio deve suportar:

Warrior  
Ranger  
Arcanist

para evitar refatoração estrutural futura.

## 11. WARRIOR

Abilities MVP:

### **Basic Attack**

Ataque básico.

### **Power Strike**

Ataque de maior dano.

### **Shield Guard**

Aumenta defesa temporariamente.

### **Dash**

Movimento curto controlado.

## 12. ABILITY DEFINITION

Uma habilidade deve ser data-driven.

AbilityId

Name

Cooldown

ResourceCost

Range

CastTime

TargetType

BaseDamage

Scaling

Effects

Nunca colocar os números diretamente no client.

## 13. GAME COMMAND

Toda ação enviada pelo client possui:

CommandId  
SessionId  
CharacterId  
Sequence  
CommandType  
Payload

Exemplo:

```
{
  "version": 1,
  "type": "Move",
  "requestId": "uuid",
  "sequence": 154,
  "payload": {
    "x": 105,
    "y": 87
  }
}
```

## 14. COMMAND SEQUENCE

Cada sessão possui sequência monotônica:

1  
2  
3  
4  
5

...

O servidor deve rejeitar:

sequence duplicada  
sequence antiga  
sequence inválida

Isso ajuda a impedir:

- replay;
- duplicação;
- comandos fora de ordem.

## 15. COMMANDS MVP

Authenticate  
SelectCharacter  
Move  
Attack  
CastAbility  
Interact  
PickupItem  
EquipItem  
UnequipItem  
UseItem  
SendChat  
CreateParty  
InviteParty  
LeaveParty

## 16. MOVEMENT

O servidor é autoridade absoluta.

Client envia intenção:

MoveIntent

O servidor valida:

tile existe  
tile caminhável  
distância máxima  
velocidade  
cooldown  
estado do personagem  
mapa atual  
teleport inválido  
sequence

O servidor decide a posição final.

Client apenas apresenta interpolação.

## 17. MAP

MVP:

Town  
Forest  
Cave

Tiles:

Ground  
Wall  
Water  
Door  
Obstacle  
Spawn  
Transition

Cada tile deve possuir:

Walkable  
BlocksVision  
BlocksProjectiles  
MovementCost

## 18. CREATURES

MVP:

Slime  
Wolf  
Spider  
Bandit  
CaveBeast

CreatureDefinition:

Id  
Name  
Level  
Health  
Mana  
Armor  
Resistances  
MovementSpeed  
AggroRange  
AttackRange  
AttackCooldown  
Damage  
ExperienceReward  
LootTable  
AiProfile

CreatureInstance:

Id  
DefinitionId  
Position

Health  
State  
Target  
SpawnPoint

## 19. CREATURE AI

Estados:

Idle  
Patrol  
Investigate  
Chase  
Attack  
Flee  
Assist  
Return  
CallForHelp  
Dead

AI tick:

5-10 Hz

Não executar AI completa a cada frame do client.

## 20. INTEREST MANAGEMENT

O servidor não deve enviar o mundo inteiro para cada jogador.

Cada jogador possui:

InterestRadius

Somente entidades relevantes são replicadas.

Exemplo:

player  
nearby players  
nearby creatures  
nearby items  
relevant NPCs  
combat effects

## 21. COMBAT

Pipeline:

ReceiveCommand  
↓  
Validate  
↓  
CheckState  
↓  
CheckRange  
↓  
CheckCooldown  
↓  
CheckTarget  
↓  
Calculate  
↓  
Apply  
↓  
GenerateEvents  
↓  
Replicate  
↓  
Persist

## 22. DAMAGE

Fórmula base:

```
RawDamage =  
AbilityBase  
+ WeaponPower  
+ AttributeScaling  
+ SkillScaling
```

Armor:

```
ArmorMultiplier =  
100 / (100 + EffectiveArmor)
```

Dano:

```
FinalDamage =  
max(  
    1,  
    RawDamage  
    × ArmorMultiplier  
    × ResistanceMultiplier  
    × CriticalMultiplier  
)
```

Todas as fórmulas devem estar atrás de interfaces:

```
IDamageCalculator  
IArmorCalculator  
ICriticalCalculator  
IResistanceCalculator
```

Isso permite testar e versionar fórmulas.

## 23. RANDOM

Random crítico:

SERVER ONLY

Random de loot:

SERVER ONLY

Random de crafting:

SERVER ONLY

Nunca utilizar RNG confiável do client.

## 24. DEATH

Pipeline:

Health  $\leq 0$

↓

Character dies

↓

Combat ends

↓

Death event

↓

Death penalty

↓

Respawn state

A penalidade final continua:

PRODUCT\_DECISION\_REQUIRED

até ser definida.

O agente não deve inventá-la.

## 25. ITEMS

Separar:

ItemDefinition  
ItemInstance

Definition:

ItemId  
Name  
Type  
Stackable  
MaxStack  
Weight  
BaseValue  
Rarity  
EquipmentSlot  
Stats

Instance:

ItemInstanceId  
ItemDefinitionId  
OwnerId  
Location  
Durability  
Modifiers  
CreatedAt

## 26. ITEM LOCATIONS

Inventory  
Equipment

World  
MarketEscrow  
TradeEscrow  
Crafting  
Destroyed

Uma instância nunca pode existir simultaneamente em dois lugares.

Essa regra deve ter teste específico.

## 27. INVENTORY

MVP:

30 slots

Suportar:

stack  
split  
move  
equip  
unequip  
destroy  
pickup

Toda operação deve ser transacional.

## 28. EQUIPMENT

Slots:

Head  
Chest  
Legs  
Feet

MainHand  
OffHand  
Accessory1  
Accessory2

Equipment deve alterar atributos efetivos.

## 29. EXPERIENCE

Interface:

IExperienceCurve

MVP:

$XP(level) = \text{floor}(100 \times level^{1.65})$

Nunca espalhar essa fórmula pelo código.

## 30. LEVEL UP

Quando XP atingir o threshold:

```
Grant XP
↓
Check threshold
↓
Level Up
↓
Apply level rewards
↓
Recalculate stats
↓
Emit LevelUpEvent
↓
Persist
```

↓

Replicate

## 31. QUESTS

Tipos:

Explore

Kill

Collect

Craft

Talk

Investigate

Escort

Puzzle

Discovery

WorldEvent

Quest state:

NotStarted

Available

Active

Completed

Failed

Objetivos devem ser data-driven.

## 32. NPC

NPC pode:

Dialogue

Quest

Shop

Crafting

Training  
Reputation

MVP:

Elder  
Blacksmith  
Healer  
Merchant  
Scout

## 33. PARTY

MVP:

maximum 5 players

Funções:

Create  
Invite  
Accept  
Leave  
Kick  
TransferLeader

Não implementar trinity rígida:

Tank  
Healer  
DPS

O grupo deve ser cooperativo sem exigir composição obrigatória.

## 34. CHAT

Canais:

Local  
Party  
Private  
System

MVP não possui chat global irrestrito.

Adicionar:

Block  
Mute  
Report  
RateLimit

## 35. WEBSOCKET

Endpoint:

`wss://game.<domain>/game`

Envelope:

```
{  
  "version": 1,  
  "type": "event",  
  "requestId": "uuid",  
  "sequence": 123,  
  "payload": {}  
}
```

Tipos:

command  
event

snapshot  
delta  
error

## 36. EVENTS

MVP:

Connected  
Authenticated  
CharacterSelected  
WorldEntered  
CharacterMoved  
CreatureSpawned  
CreatureMoved  
CombatStarted  
DamageApplied  
CreatureDied  
LootDropped  
ItemPickedUp  
ItemEquipped  
ExperienceGained  
LevelUp  
CharacterDied  
CharacterRespawned  
ChatMessage  
PartyUpdated  
WorldSnapshot  
WorldDelta

## 37. ERROR CODES

Criar enum:

Unauthorized  
InvalidCommand

InvalidSequence  
CharacterNotFound  
CharacterNotInWorld  
InvalidPosition  
TileBlocked  
OutOfRange  
CooldownActive  
InsufficientResource  
InvalidTarget  
InventoryFull  
ItemNotFound  
InvalidItemState  
PartyFull  
RateLimited  
ServerBusy  
InternalError

Nunca enviar stack trace para o client.

## 38. SESSION

GameSession

Campos:

SessionId  
AccountId  
CharacterId  
ConnectionId  
ConnectedAt  
LastHeartbeat  
LastSequence  
State

## 39. RECONNECT

Disconnect:

Client disconnects

↓

Session enters DisconnectGrace

↓

Character remains server-owned

↓

30 seconds

↓

Reconnect?

YES → restore

NO → persist safe state

Durante DisconnectGrace:

não aceitar comandos externos

não duplicar sessão

não permitir login paralelo

## 40. HEARTBEAT

Client:

ping every 20 seconds

Server:

timeout after 60 seconds

O WebSocket middleware também pode possuir keep-alive próprio. A documentação do ASP.NET Core recomenda mecanismos de timeout/ping para lidar com desconexões abruptas, pois uma perda de rede pode não produzir imediatamente uma mensagem de disconnect.

## 41. GAME LOOP

MVP:

20 Hz

Responsabilidades:

process commands  
movement  
combat  
effects  
world updates  
replication scheduling

AI:

5 Hz

Não amarrar o loop ao frame rate do client.

## 42. PERSISTENCE

Não persistir cada frame.

Persistir em eventos importantes:

login  
logout  
movement checkpoint  
item mutation  
equipment mutation  
combat result  
XP  
level  
quest progression

currency mutation  
death  
trade  
market

## 43. POSTGRESQL

Tabelas iniciais:

accounts  
characters  
character\_attributes  
character\_skills  
character\_equipment  
items  
inventories  
wallets  
transactions  
quests  
quest\_states  
parties  
party\_members  
chat\_messages  
audit\_events  
outbox\_messages  
sessions

## 44. CONCURRENCY

Operações críticas devem usar transação.

Principalmente:

Gold  
Inventory  
Trade

Market  
Equipment  
Loot  
Crafting

Nunca:

read balance  
calculate  
write balance

sem proteção transacional/concurrency adequada.

## 45. REDIS

Redis pode armazenar:

session state  
presence  
distributed locks  
rate limits  
temporary cache  
transient world data

Redis NÃO é source of truth para:

Gold  
Items  
XP  
Character ownership  
Market ownership  
Transactions

## 46. OUTBOX

Eventos persistentes devem usar:

Domain Event

↓

Outbox

↓

Worker

↓

External processing

Exemplos:

Notification

Analytics

Email

Audit integration

## 47. AUTH

Fluxo:

POST /auth/register

POST /auth/login

POST /auth/refresh

Após login:

AccessToken

RefreshToken

GameToken

GameToken deve possuir:

AccountId

CharacterId

Expiration  
TokenId

O GameServer nunca deve confiar apenas em:

CharacterId enviado pelo client

O CharacterId precisa ser associado à sessão autenticada.

## 48. API

Endpoints iniciais:

GET /health

GET /ready

POST /auth/register

POST /auth/login

POST /auth/refresh

GET /characters

POST /characters

POST /characters/{id}/select

GET /content/version

Admin ficará separado.

## 49. GODOT

Estrutura:

client/Ether.Game/

scenes/

bootstrap/

```
    auth/  
    character/  
    world/  
    ui/  
  
scripts/  
    network/  
    world/  
    entities/  
    combat/  
    ui/  
    input/  
    camera/  
  
resources/  
    items/  
    creatures/  
    abilities/  
    maps/
```

## 50. CLIENT NETWORK LAYER

Criar:

```
NetworkClient  
ProtocolSerializer  
CommandSender  
EventDispatcher  
SnapshotProcessor  
DeltaProcessor  
ReconnectManager  
HeartbeatManager
```

O client deve possuir uma camada de abstração:

```
GameClient
```

para não espalhar WebSocket pelo projeto.

## 51. MOBILE INPUT

Suportar:

Tap-to-move

Virtual joystick

Tap target

Context action

Action bar

Prioridade:

one-hand playable

Duas mãos devem aumentar eficiência, não ser obrigatórias.

## 52. CAMERA

MVP:

Top-down

Com leve sensação isométrica por arte/iluminação.

Camera:

follow player

bounded by map

smooth interpolation

## 53. RENDERING

MVP:

2D HD stylized

Prioridades:

clarity  
readability  
combat feedback  
mobile performance

Não tentar produzir centenas de efeitos simultaneamente.

## 54. SNAPSHOT

Ao entrar no mundo:

WorldSnapshot

Inclui:

player  
nearby players  
creatures  
items  
NPCs  
map information  
current state

Depois:

WorldDelta

somente alterações.

## 55. CONTENT PIPELINE

Conteúdo deve ser data-driven.

Exemplo:

```
{  
  "id": "creature.slime",  
  "name": "Slime",  
  "level": 1,  
  "health": 35,  
  "armor": 2,  
  "movementSpeed": 1.5,  
  "experienceReward": 12  
}
```

O servidor valida todos os arquivos antes de carregá-los.

## 56. CONTENT VALIDATOR

Validar:

- duplicate IDs
- invalid references
- negative values
- invalid loot tables
- missing abilities
- missing NPC references
- invalid quest references
- invalid maps
- invalid item references

Falha de content validation deve impedir startup em ambiente de produção.

## 57. SEED MVP

### Town

Town Square  
Inn  
Blacksmith  
Merchant  
Healer  
Elder  
North Gate  
South Gate

### Forest

Forest Entrance  
Clearing  
River  
Old Shrine  
Wolf Den  
Hidden Path

### Cave

Cave Entrance  
Upper Tunnel  
Crystal Room  
Bandit Camp  
Deep Chamber

## 58. FIRST ITEMS

Rusty Sword  
Wooden Shield  
Traveler's Bow

Cloth Hood  
Leather Tunic  
Leather Boots  
Health Potion  
Mana Potion  
Copper Ore  
Iron Ore  
Herb  
Wolf Pelt  
Spider Silk  
Slime Gel

## 59. FIRST QUESTS

### **A New Beginning**

Tutorial.

### **Trouble in the Forest**

Investigation/combat.

### **The Missing Shipment**

Collection/exploration.

### **Into the Cave**

Combat/exploration.

### **The Old Shrine**

Discovery.

## 60. FIRST 5 MINUTES

O jogador deve conseguir:

login  
create character  
enter world  
move  
see another player  
talk to NPC  
receive first quest  
kill first creature  
loot item  
equip item  
gain XP

Nenhuma tela desnecessária.

## 61. FIRST 30 MINUTES

Até 30 minutos o jogador deve experimentar:

exploration  
combat  
loot  
equipment  
progression  
quest  
NPC interaction  
risk  
death/recovery  
social presence

Não obrigar o jogador a:

assistir tutorial longo  
comprar moeda  
abrir dezenas de menus  
fazer daily  
esperar energia  
usar autoplay

## 62. ADMIN

Criar inicialmente apenas infraestrutura.

Endpoints administrativos:

```
inspect character
inspect account
inspect inventory
inspect wallet
inspect transactions
inspect sessions
inspect audit
```

Toda operação administrativa deve gerar:

AuditEvent

## 63. AUDIT EVENT

Campos:

```
AuditEventId
Timestamp
ActorType
ActorId
Action
TargetType
TargetId
Payload
CorrelationId
```

Exemplos:

```
ADMIN_GRANT_ITEM
ADMIN_GRANT_GOLD
```

ADMIN\_TELEPORT  
PLAYER\_TRADE  
MARKET\_ORDER\_CREATED  
ITEM\_DESTROYED  
CHARACTER\_DIED

## 64. OBSERVABILITY

Implementar:

OpenTelemetry  
structured logging  
correlation ID  
request ID  
session ID  
character ID

Métricas:

active\_sessions  
commands\_per\_second  
command\_latency  
combat\_latency  
database\_latency  
redis\_latency  
websocket\_connections  
reconnects  
disconnects  
errors  
ticks  
tick\_duration

## 65. SECURITY

Testar:

speed hack  
teleport  
invalid movement  
invalid target  
invalid item ownership  
gold duplication  
item duplication  
replay command  
sequence replay  
double login  
session hijack  
rate limit bypass  
invalid WebSocket token

## 66. ECONOMY INVARIANTS

Obrigatório:

Gold nunca negativo.  
ItemInstance nunca possui dois owners simultâneos.  
Trade não pode criar item.  
Market não pode duplicar item.  
Loot somente pode ser criado pelo servidor.  
Client nunca pode definir preço final diretamente.  
Transaction precisa possuir origem.

## 67. TRANSACTION TYPES

QuestReward  
CreatureLoot  
VendorPurchase  
VendorSale  
PlayerTrade  
MarketSale  
MarketTax  
CraftingCost

AdminAdjustment  
DeathPenalty

## 68. TESTES OBRIGATÓRIOS

### Domain

Testar:

Gold  
Character  
Attributes  
Level  
XP  
Damage  
Death  
Equipment  
Inventory

### Application

Testar:

CreateCharacter  
MoveCharacter  
AttackCreature  
PickupItem  
EquipItem  
QuestProgression  
Party

### Infrastructure

Testar:

EF mappings  
transactions

concurrency  
outbox  
repositories

## GameServer

Testar:

commands  
sequence  
sessions  
movement  
combat  
AI  
replication  
reconnect

## 69. CRITICAL INVARIANT TESTS

Criar testes explicitamente para:

cannot move through wall  
cannot move faster than allowed  
cannot attack from invalid range  
cannot use dead character  
cannot loot another player's restricted item  
cannot duplicate item  
cannot duplicate gold  
cannot replay command  
cannot maintain two authoritative sessions  
cannot equip incompatible item  
cannot spend nonexistent gold

# 70. INTEGRATION TEST

Cenário obrigatório:

Create account

↓

Login

↓

Create Warrior

↓

Select Warrior

↓

Connect WebSocket

↓

Enter Town

↓

Move

↓

Enter Forest

↓

Find Slime

↓

Attack

↓

Kill Slime

↓

Receive XP

↓

Receive Loot

↓

Equip Item

↓

Logout

↓

Login

↓

Verify persistence

Esse teste representa o primeiro vertical slice.

## 71. MULTIPLAYER TEST

Criar:

Player A

Player B

Ambos:

login

enter same map

see each other

chat

party

fight same creature

Validar que ambos recebem estados consistentes.

## 72. CRASH TEST

Durante:

combat

loot

equipment

trade

simular disconnect/crash.

Após reconnect:

state must be consistent

Nenhum:

item duplication

gold duplication

XP duplication

## 73. DOCKER COMPOSE

Serviços:

```
postgres
redis
api
game-server
worker
```

Volumes:

```
postgres-data
redis-data
```

Health checks obrigatórios.

## 74. DEVELOPMENT COMMANDS

README deve possuir:

```
dotnet restore
dotnet build
dotnet test
docker compose up -d
docker compose down
```

Também:

```
dotnet ef database update
```

e comandos de seed.

## 75. CI

Pipeline:

```
restore
build
test
architecture tests
content validation
docker build
```

Nenhum PR deve passar se:

```
build fails
tests fail
architecture rules fail
content validation fails
```

## 76. ARCHITECTURE TESTS

Regras:

```
Domain must not depend on Infrastructure
Domain must not depend on API
Domain must not depend on Godot
```

Application must not depend on Infrastructure

Contracts must remain dependency-light

GameServer cannot reference Godot

Infrastructure cannot leak into Domain

# 77. PRIMEIRA ORDEM DE IMPLEMENTAÇÃO

O agente deve executar nesta ordem:

ETHER-001 Repository  
ETHER-002 Solution  
ETHER-003 Domain  
ETHER-004 Contracts  
ETHER-005 Application  
ETHER-006 Infrastructure  
ETHER-007 API  
ETHER-008 GameServer  
ETHER-009 Worker  
ETHER-010 Tests

Depois:

ETHER-011 PostgreSQL  
ETHER-012 Redis  
ETHER-013 Docker  
ETHER-014 EF Core  
ETHER-015 Migrations  
ETHER-016 Seed

Depois:

ETHER-020 Account  
ETHER-021 Authentication  
ETHER-022 Character  
ETHER-023 Character persistence  
ETHER-024 GameToken  
ETHER-025 WebSocket  
ETHER-026 Session

Depois:

ETHER-030 World  
ETHER-031 Map

ETHER-032 Movement  
ETHER-033 Interest management

Depois:

ETHER-040 Creatures  
ETHER-041 AI  
ETHER-042 Combat  
ETHER-043 Loot  
ETHER-044 Death

Depois:

ETHER-050 Inventory  
ETHER-051 Equipment  
ETHER-052 XP  
ETHER-053 Level

Depois:

ETHER-060 Quests  
ETHER-061 NPC  
ETHER-062 Party  
ETHER-063 Chat

Depois:

ETHER-070 Replication  
ETHER-071 Snapshot  
ETHER-072 Delta  
ETHER-073 Reconnect

Depois:

ETHER-080 Godot bootstrap  
ETHER-081 Login  
ETHER-082 Character creation  
ETHER-083 World  
ETHER-084 Movement

## 78. PRIMEIRO MILESTONE

O primeiro milestone é:

M0 – SERVER BOOTS

Critério:

`dotnet build`

PASS

`dotnet test`

PASS

`docker compose up`

PASS

`GET /health`

200

PostgreSQL

READY

Redis

READY

## 79. SEGUNDO MILESTONE

M1 – FIRST CHARACTER

Deve permitir:

`register`

`login`

`create Warrior`

`select Warrior`

`persist Warrior`

`logout`

login  
retrieve Warrior

## 80. TERCEIRO MILESTONE

M2 – FIRST WORLD

Deve permitir:

WebSocket  
authentication  
spawn  
movement  
map validation  
snapshot  
delta  
disconnect  
reconnect

## 81. QUARTO MILESTONE

M3 – FIRST COMBAT

Deve permitir:

find Slime  
target Slime  
attack  
damage  
kill  
XP  
loot

## 82. QUINTO MILESTONE

### M4 – FIRST PROGRESSION

Deve permitir:

loot  
inventory  
equipment  
XP  
level  
death  
respawn

## 83. SEXTO MILESTONE

### M5 – FIRST SOCIAL

Deve permitir:

two players  
chat  
party  
shared world  
shared combat

## 84. SÉTIMO MILESTONE

### M6 – FIRST QUEST

Deve permitir:

NPC  
dialogue  
quest  
objective  
combat  
reward  
completion

## 85. VERTICAL SLICE COMPLETO

O primeiro verdadeiro objetivo do projeto é:

LOGIN  
↓  
CHARACTER  
↓  
TOWN  
↓  
FOREST  
↓  
CREATURE  
↓  
COMBAT  
↓  
LOOT  
↓  
INVENTORY  
↓  
EQUIPMENT  
↓  
XP  
↓  
QUEST  
↓  
DEATH  
↓  
RESPAWN  
↓  
LOGOUT

↓  
LOGIN  
↓  
PERSISTED STATE

E posteriormente:

PLAYER A  
    ↘  
        SHARED WORLD  
    ↗  
PLAYER B

## 86. NÃO IMPLEMENTAR AINDA

O agente deve explicitamente deixar fora:

Guild Wars  
Housing  
Mounts  
Pets  
Open PvP  
World Boss  
Gacha  
Battle Pass  
Premium Shop  
Subscription  
Territory Control  
Large-scale PvP  
Advanced Market  
Advanced Crafting  
Ranger  
Arcanist

Esses sistemas somente entram depois da validação do core loop.

## 87. DEFINIÇÃO DE DONE

Uma tarefa só pode ser marcada como concluída quando:

Código implementado  
+  
Testes adicionados  
+  
Build passando  
+  
Testes passando  
+  
Regras documentadas  
+  
Nenhuma violação arquitetural  
+  
Nenhum SPEC\_GAP escondido

## 88. RELATÓRIO OBRIGATÓRIO DO AGENTE

Ao terminar cada milestone:

MILESTONE:

STATUS:

BASELINE:

COMMIT:

IMPLEMENTED:

- ...  
- ...

TESTS:

- ...  
- ...

BUILD:

PASS/FAIL

ARCHITECTURE:

PASS/FAIL

DATABASE:

PASS/FAIL

SECURITY:

PASS/FAIL

SPEC\_GAPS:

- NONE

ou

- ...

SPEC\_CONFLICTS:

- NONE

ou

- ...

PRODUCT\_DECISIONS\_REQUIRED:

- NONE

ou

- ...

NEXT:

- ...

## 89. PROMPT MESTRE PARA OPENCODE

O agente que receber este documento deve iniciar com:

Você é o principal engenheiro responsável pela implementação do Project Ether.

Você recebeu o Blueprint v5.0 completo.

Sua responsabilidade não é apenas escrever código: é transformar o blueprint em um sistema executável, testável, observável e evolutivo.

Antes de implementar:

1. inspecione o repositório;
2. determine se existe código prévio;
3. preserve qualquer código útil;
4. não destrua trabalho existente;
5. identifique divergências;
6. crie um plano incremental.

Depois implemente na ordem definida pelo blueprint.

Não implemente features fora do milestone atual.

Não invente regras de negócio.

Quando faltar uma decisão, registre:

`SPEC_GAP`

Quando houver conflito:

`SPEC_CONFLICT`

Quando depender de uma decisão de produto:

`PRODUCT_DECISION_REQUIRED`

Nunca esconda essas situações.

Toda regra crítica precisa possuir teste.

Toda alteração de estado crítico precisa ser server-authoritative.

Gold, inventory, items, XP, ownership, combat, movement, loot e progression jamais podem depender da decisão do client.

O client é uma apresentação + envio de intenções.

O servidor é a autoridade.

Implemente primeiro o M0.

Não avance para M1 até:

build PASS  
tests PASS  
architecture PASS  
docker PASS  
health PASS

Depois avance sequencialmente.

Ao finalizar cada milestone, produza o relatório definido no Blueprint.

Não considere uma tarefa concluída apenas porque o código compila.

O objetivo é construir uma base sobre a qual o MMORPG possa crescer sem precisar ser reescrito.

## 90. PRIMEIRO COMANDO DO AGENTE

Depois de fornecer o Blueprint ao agente, o primeiro comando deve ser:

Leia todo o Blueprint Project Ether v5.0.

Não implemente nada ainda.

Primeiro:

1. inspecione o repositório;
2. liste arquivos existentes;
3. identifique solução/projetos;
4. identifique tecnologias;
5. identifique banco;
6. identifique infraestrutura;
7. identifique código reaproveitável;
8. identifique conflitos com o Blueprint;
9. produza um diagnóstico;
10. proponha o plano M0.

Não faça alterações neste passo.

Aguarde somente depois de produzir o diagnóstico.

Depois do diagnóstico:

Agora implemente M0 exatamente conforme o Blueprint.

Crie a solução completa.

Execute build.

Execute testes.

Execute architecture tests.

Suba Docker Compose.

Valide PostgreSQL.

Valide Redis.

Valide /health.

Corrija todos os problemas encontrados.

Não avance para M1.

Ao final produza o relatório M0.

## 91. PRINCÍPIO DE DESENVOLVIMENTO

O projeto deve crescer nesta direção:

FOUNDATION



SERVER



WORLD



COMBAT



PROGRESSION

↓  
SOCIAL  
↓  
ECONOMY  
↓  
CONTENT  
↓  
LIVE OPS

Nunca:

SHOP  
↓  
GACHA  
↓  
BATTLE PASS  
↓  
200 MAPS  
↓  
500 ITEMS  
↓  
GAMEPLAY

## 92. REGRA MAIS IMPORTANTE DO PROJECT ETHER

O objetivo não é criar:

"um jogo com muitas features"

O objetivo é criar:

"um mundo no qual o jogador queira voltar"

Portanto, qualquer feature futura deve responder:

Isso torna o mundo mais interessante?  
Isso cria uma decisão significativa?  
Isso cria uma razão para explorar?  
Isso cria uma razão para interagir?  
Isso cria uma razão para dominar o sistema?  
Isso cria uma razão legítima para voltar amanhã?

Se a resposta for não para todas:

FEATURE DEVE SER QUESTIONADA.

## 93. ESTADO FINAL DESTA VERSÃO

Ao concluir este blueprint, o agente possui especificação suficiente para começar a construir:

```
.NET solution
+
Domain
+
Application
+
Infrastructure
+
API
+
GameServer
+
Worker
+
PostgreSQL
+
Redis
+
WebSocket
+
Godot client
+
Content pipeline
+
```

Tests

+

Docker

+

CI

+

Observability

A próxima etapa não deve ser outro documento conceitual.

Deve ser:

M0 → código real

e, depois:

M1 → primeiro personagem

M2 → primeiro mundo

M3 → primeiro combate

M4 → primeira progressão

M5 → primeiro multiplayer

M6 → primeira quest

**Esse é o ponto em que Project Ether deixa de ser uma ideia e começa a existir como software.**